Public editions adapted from real Career Agent Pro outputs

Edição pública adaptada de uma saída real do produto. Material educacional gerado por IA e revisado para publicação. Todos os contextos de arquitetura, equipes, incidentes, requisitos e avaliação são hipóteses didáticas de plataformas fictícias. Não descrevem informações internas confirmadas, questões oficiais, contratação ou estratégia de empresas reais. Sem vínculo ou endosso de Prezensa, Globo, WhatsApp ou Meta.

System design Android: mensageria e pagamentos

Liderança Android e sistemas financeiros para plataforma fictícia de mensageria. Todos os parâmetros são escolhas didáticas, não dados de uma empresa real.

Liderança técnica: plataforma fictícia de mensageria

> **Como usar este material:** Cada desafio simula uma sessão real de entrevista técnica no nível Staff/Senior. Estude os trade-offs com atenção - eles são o coração da avaliação. O facilitador da simulação não busca a resposta certa, mas evidência de raciocínio estruturado diante de decisões com consequências concretas.

Desafio 1 - Arquitetura Android para Pagamentos P2P em Escala

Contexto

Cenário fictício: uma plataforma de mensageria estuda transferências P2P, pagamentos a negócios e uma futura área de comércio. Defina uma carga sintética e limites de capacidade; não se atribui base de usuários ou roadmap a empresa real.

Hipótese técnica: na base fictícia, pagamentos estão acoplados à UI de chat e os limites de domínio e recuperação de falhas ainda são insuficientes. Proponha evolução incremental, com contratos explícitos e testes de falha, sem reescrever mensageria.

Sua missão: como Staff Engineer, definir a arquitetura do cliente Android para essa expansão. **Formato:** 45 minutos. O facilitador da simulação representa o papel de Engineering Manager do time.

Requisitos Funcionais

- Transferência Pix iniciada a partir de uma conversa, da lista de contatos ou de um fluxo dedicado de pagamentos
- QR dinâmico no contrato simulado do provedor; validar formato e semântica na integração real, sem tratar este exercício como especificação Pix

- Histórico de transações com estados auditáveis: `iniciado`, `processando`, `confirmado`, `falhou`, `revertido`
- Notificações best-effort com estado consultável e reconciliação independente; ausência de push não determina falha financeira.
- Suporte a múltiplas chaves Pix por usuário

Requisitos Não Funcionais

Dimensão	Requisito
Dispositivos	Alvo sintético do exercício: 2 GB de RAM e Android 8+. Medir memória, startup e falhas nessa matriz; não é estatística de base brasileira.
Conectividade	Lidar com quedas de rede no meio de transações sem perda silenciosa de estado
Latência	Feedback visual em menos de 300ms após iniciar uma ação, mesmo com transação em processamento
Correctness	Uma transação nunca pode ser processada duas vezes por falha do cliente
Auditabilidade	Todos os estados de transação persistidos localmente com timestamp para auditabilidade do exercício; validar retenção e minimização
Testabilidade	Arquitetura que permita desenvolvimento paralelo entre os 8 engenheiros do time

---

Desafios Ocultos

> **Atenção:** Estes são os pontos onde o facilitador da simulação vai pressionar - e onde a maioria dos candidatos tropeça. Prepare respostas sólidas para cada um.

---

① O problema de idempotência no cliente

Retry automático em queda de rede parece óbvio. A pergunta que vem em seguida é: *como você garante que o retry não processa a transação duas vezes?*

A resposta exige pensar em **idempotency keys geradas no cliente**, persistidas antes do envio, e uma estratégia de reconciliação com o servidor.

---

② A tensão entre offline-first e correctness financeiro

Offline-first é boa prática para apps convencionais. Para pagamentos, é perigoso - você não pode confirmar uma transação que o servidor não processou.

O candidato precisa distinguir:

- Pode ser offline-first: histórico de transações, interface

- Não confirmar nem executar pagamento offline; uma intenção local pode ser enfileirada, expirar e exigir confirmação antes de submissão online

---

③ O estado intermediário da transação

O que acontece quando o app é forçado a fechar no meio de uma transação?

Neste modelo de falhas, persistir a intenção antes da rede facilita recuperação. Compare essa decisão com custos de retenção, expiração e reconciliação; persistência local não confirma execução financeira.

---

④ Modularização sem over-engineering

Uma proposta de modularização excessiva resolve problemas teóricos, mas cria complexidade real de build e ownership.

Espere perguntas como: *Quantos módulos você criaria? Quem é dono de cada um? Como você gerencia dependências entre eles?*

---

⑤ O acoplamento com o cliente de mensageria

O sistema de pagamentos precisa conviver com o sistema de mensagens - compartilhando infraestrutura de rede, notificações e banco de dados local. A resposta não pode incluir reescrever o app de mensagens.

Perguntas para simulação

Abertura:

> *"Me descreve como você estruturaria a arquitetura Android para esse sistema de pagamentos. Pode começar alto nível - vamos aprofundando."*

Aprofundamento técnico:

> *"Você mencionou que vai persistir o estado da transação localmente. Me mostra exatamente qual é esse modelo de dados e em que momento você persiste cada estado."*

> *"O usuário está no meio de uma transferência de R$ 500 e a rede cai. Me descreve passo a passo o que acontece no cliente - do momento da queda até o momento que o usuário reabre o app."*

> *"Como você garante que um retry automático não resulta em débito duplo na conta do usuário?"*

Pressão em trade-offs:

> *"Você está propondo separar o módulo de pagamentos do de mensageria. Qual é o custo real disso em build time, ownership e complexidade de integração? Vale a pena nesse momento do time?"*

> *“Você usaria Room ou outra solução de persistência para o estado das transações? Por quê? Quais os trade-offs?”*

Pressão em contexto real:

> *“Em dispositivos com 2 GB de RAM e Android 8, como sua arquitetura se comporta? Onde você espera pressão de memória? Como você diagnosticaria um problema em produção nesse cenário?”*

> *“O time de backend mudou o contrato da API de pagamentos sem avisar. Como sua arquitetura isola o cliente Android dessa mudança?”*

Liderança técnica:

> *“Você tem 8 engenheiros com níveis diferentes. Como você comunicaria essa arquitetura para o time? Como garantiria que todos a seguissem sem você se tornar um gargalo de code review?”*

Alternativas e trade-offs

> **Dica estratégica:** O facilitador da simulação não está procurando a resposta certa. Está procurando evidência de que você pensa em trade-offs reais com consequências concretas.

Trade-off 1 - Modularização vs. velocidade de desenvolvimento

Opção	Vantagem	Custo
Módulo dedicado de pagamentos desde o início	Isolamento de domínio, testabilidade, ownership claro	Build time maior, overhead de setup, complexidade de DI entre módulos
Feature package dentro do módulo existente	Velocidade imediata, sem overhead de configuração	Risco de acoplamento crescente, dificuldade de teste isolado

Abordagem possível e limites: começar com separação clara de pacotes e interfaces de contrato dentro do módulo existente - o que permite extração para módulo separado quando o time crescer, sem criar complexidade antes de precisar dela.

Trade-off 2 - Persistência de estado: antes ou depois da chamada de rede

Opção	Vantagem	Custo
Persistir ANTES de chamar a API	Facilita recuperação, condicionada à durabilidade e reconciliação	Complexidade de reconciliação com servidor
Persistir APÓS confirmação do servidor	Estado local reflete realidade do servidor	Perde contexto se o app fechar antes da resposta chegar

Abordagem possível e limites: persistir a intenção da transação com estado `pendente` antes da chamada, com idempotency key gerada no cliente. Após resposta do servidor, atualizar para o estado final. Em caso de ambiguidade, reconciliar ativamente na próxima sessão - nunca assumir falha silenciosa.

Trade-off 3 - Retry automático vs. confirmação explícita do usuário

Opção	Vantagem	Custo
Retry automático com backoff	UX mais fluida, sem ação do usuário	Risco de duplo processamento se idempotência falhar
Confirmação explícita para retry	Controle total do usuário, ainda requer identidade estável e deduplicação no servidor	Atrito na UX

Abordagem possível: reenviar a mesma intenção com a mesma chave, usando backoff com jitter e limites. O servidor simulado deve deduplicar atomicamente por identidade/escopo da intenção e devolver resultado consistente. Timeout não prova falha: persistir estado incerto e reconciliar por consulta, na sessão ativa ou quando o SO permitir recuperação.

Trade-off 4 - ViewModel compartilhado vs. ViewModels por tela

Opção	Vantagem	Custo
ViewModel compartilhado no fluxo de pagamento	Estado consistente entre telas, sem re-fetch	Lifetime management complexo, dificulta testabilidade individual
ViewModels independentes por tela	Isolamento, testabilidade	Risco de inconsistência de estado, re-fetches desnecessários

Abordagem possível e limites: ViewModel compartilhado com escopo de navigation graph para o fluxo de pagamento, com estado imutável publicado via `StateFlow`. Cada tela observa apenas o estado relevante para ela - sem duplicação, sem acoplamento direto entre ViewModels.

Desafio 2 - Sistema de Notificações de Alta Confiabilidade para Transações Financeiras

Contexto

Incidente inteiramente fictício: num protótipo de pagamentos, algumas confirmações visuais não aparecem quando o app está em background. Investigue separadamente estado autoritativo da transação, transporte de notificações, permissões e apresentação. Não é um relato de incidente em produto real.

Premissa do exercício: um backend simulado emite notificações via FCM. O diagnóstico ainda é aberto e deve considerar cliente, servidor e transporte; nenhum componente é declarado culpado antecipadamente.

Sua missão: definir a estratégia de notificações de transação no cliente Android.

Requisitos Funcionais

- Meta sintética: buscar apresentação em até 10 segundos quando processamento, rede, permissão e SO permitirem. Medir percentis; não é garantia universal de entrega.
- Objetivo condicionado: apresentar confirmação quando rede, permissões e SO permitirem. Em indisponibilidade, persistir intenção e reconciliar estado; não prometer push universal.
- Na sessão ativa, iniciar consulta de estado após 30 segundos sem atualização, quando houver rede e autorização. Em background, agendar recuperação quando o SO permitir; exibir estado incerto sem declarar falha.
- Parâmetro didático: discutir retenção de até 90 dias, minimização, limpeza e isolamento por conta; não é obrigação regulatória nem recomendação universal de retenção.
- Em modo de economia de bateria, o comportamento deve ser degradado de forma **transparente** - nunca silenciosa

---

Requisitos Não Funcionais

Dimensão	Requisito
Fragmentação Android	Comportamento de background processing varia drasticamente entre fabricantes (Xiaomi, Samsung, Motorola) - especialmente no Android 8-11
Consumo de bateria	A solução não pode drenar bateria perceptivelmente - usuários de dispositivos de entrada são sensíveis a isso
Confiabilidade	Detectar estado incerto e recuperá-lo por consulta; monitorar atraso e duplicação sem prometer notificação universal
Privacidade	Valor e destinatário não podem aparecer em texto claro na notificação com dispositivo bloqueado

---

Desafios Ocultos

> **Atenção:** Estes são os pontos de pressão do facilitador da simulação. Respostas superficiais serão desafiadas.

---

① A armadilha do FCM como solução completa

FCM não oferece prazo determinístico de entrega. Prioridade, conectividade, permissões, estados de energia e política da plataforma afetam processamento e apresentação. Alta prioridade é para conteúdo urgente e visível ao usuário, pode ser rebaixada e não substitui reconciliação de estado.

A solução exige camada de redundância.

---

② Doze Mode e App Standby Buckets

O Android tem mecanismos progressivos de restrição de background processing que a maioria dos candidatos ignora ou menciona superficialmente.

Espere perguntas como: *Como sua solução se comporta quando o app está no bucket "rare" do App Standby?*

---

③ **Notificação de UI ≠ garantia de estado**

Notificar o usuário é diferente de garantir que o estado da transação foi atualizado localmente. Essas responsabilidades precisam ser separadas:

- Notificação visual → **best-effort**
- Sincronização de estado → **obrigatória, com mecanismo próprio**

---

④ **Fragmentação de OEM - o elefante na sala**

④ Matriz de dispositivos: valide combinações sintéticas de API, permissões, fabricante, energia e rede. Instrumente recebimento, atualização de estado e exibição separadamente. Não se assume market share ou taxa de falha real por fabricante.

O candidato precisa demonstrar consciência explícita disso.

---

⑤ **WorkManager não é bala de prata**

WorkManager agenda trabalho persistente sob constraints e decisões do sistema. PeriodicWorkRequest tem intervalo mínimo de 15 minutos; isso não é prazo garantido de execução nem janela fixa de Doze. OneTimeWorkRequest pode ter atraso mínimo/backoff, mas também não assegura polling em 30 segundos.

> **Erro comum:** propor WorkManager como solução única sem reconhecer essas limitações.

Perguntas para simulação

Abertura:

> Como reduzir tempo até confirmação e permitir consulta/reconciliação quando push não chega? Explique condições de funcionamento, limites de background e indicadores de atraso.

Aprofundamento técnico:

> *"Você mencionou FCM. Me explica o que acontece exatamente quando o dispositivo está em Doze Mode e chega uma notificação FCM de alta prioridade."*

> Como a prioridade definida na mensagem enviada pelo servidor afeta FCM? Quais condições podem atrasar, rebaixar ou impedir apresentação, e como distinguir prioridade de garantia de entrega?

> *"Como você implementaria o mecanismo de fallback quando o FCM não entrega em 30 segundos? O que acontece no código?"*

Pressão em fragmentação:

> Na matriz fictícia, teste diferentes aparelhos e estados de energia. Como a solução identifica atraso, falta de permissão e restrição de background sem supor comportamento uniforme de um fabricante?

> *“Como você detecta em produção que o mecanismo de notificação está falhando em um subconjunto de dispositivos? Quais métricas você instrumentaria?”*

Pressão em trade-offs:

> *“Notificação com valor e destinatário em texto claro vs. notificação genérica por privacidade. Qual você escolhe e por quê? Quais são as implicações de compliance?”*

> *“WorkManager vs. JobScheduler vs. ForegroundService para o fallback de sincronização. Me justifica a escolha.”*

Impacto real:

> *“Como você mediria o sucesso dessa solução? Como você diferencia falha de entrega do FCM de falha no seu mecanismo de fallback?”*

Alternativas e trade-offs

Trade-off 1 - FCM alta prioridade sempre vs. prioridade adaptativa

Opção	Vantagem	Custo
Sempre usar `high priority`	Máxima chance de acordar o dispositivo	Maior consumo e possibilidade de rebaixamento; não tratar alta prioridade como forma de contornar políticas de entrega
Prioridade alta apenas para eventos urgentes e visíveis ao usuário	Uso responsável de recursos, sustentável em produção	Complexidade de categorização, risco de classificação errada

Usar alta prioridade para mensagens realmente urgentes e visíveis ao usuário, respeitando permissões e políticas FCM. Mensagens apenas para sincronização não devem depender de alta prioridade como bypass do sistema; monitorar rebaixamento, entrega e apresentação separadamente.

Trade-off 2 - Polling periódico vs. event-driven com fallback

Opção	Vantagem	Custo
Polling periódico de transações pendentes	Simples, previsível, independe de push	Consumo de bateria e rede, latência proporcional ao intervalo
Event-driven (FCM) com fallback de polling	Eficiente, baixa latência no caminho feliz	Dois mecanismos para manter, maior complexidade

FCM sinaliza mudança de estado. Na sessão ativa, um temporizador pode iniciar reconciliação após 30 segundos sem atualização, sob rede disponível. Em background, WorkManager recupera pendências de forma adiada, sem SLA de 30 segundos. Deduplicar eventos, limitar retries e consultar o servidor ao reabrir o app.

Trade-off 3 - Notificação com conteúdo completo vs. notificação genérica

Opção	Vantagem	Custo
Valor e destinatário em texto claro	UX melhor, informação imediata	Expõe dados financeiros na tela de bloqueio, conflito potencial com regulação
Notificação genérica (“atualização de pagamento”)	Menor exposição de dados no lockscreen; ainda depende de configuração, permissões e políticas de retenção	UX degradada, usuário precisa abrir o app
Notificação adaptativa por estado do dispositivo	Melhor dos dois mundos	Complexidade com `NotificationVisibility` e `KeyguardManager`

Uma opção é VISIBILITY_PRIVATE com publicVersion genérica, respeitando permissões e preferências de lockscreen. Uma checagem pontual de KeyguardManager não garante sigilo em todas as transições. Não incluir valores/destinatários sensíveis no payload de demonstração e validar a política de exposição com responsáveis pelo produto.

Desafio 3 - Estratégia de Modularização do Aplicativo Fictício de Mensageria

Contexto

Cenário fictício: um app de mensageria concentra responsabilidades num módulo predominante. Neste exercício, há builds lentos e equipes desejam reduzir acoplamento entre Messaging, Payments e Commerce. Confirme métricas e dependências antes de propor extração de módulos.

Sua missão: como Staff Engineer, apresentar a estratégia de modularização com horizonte de 18 meses.
Formato: 30 minutos de apresentação e defesa da proposta perante o CTO.

> Este desafio contém ambiguidade deliberada. Faça perguntas sobre capacidade da equipe, dependências e métricas de build; registre hipóteses antes de apresentar um plano de migração.

Requisitos Funcionais

- Times de Messaging, Payments e Commerce devem desenvolver e fazer release de features de forma independente
- Features de Payments devem ser testáveis em isolamento completo, sem depender da infraestrutura de messaging

- Suporte a feature flags granulares por módulo para rollout gradual
- Orçamento didático: limitar aumento de APK e medir impacto por módulo e variante. Não é requisito empresarial real.

Requisitos Não Funcionais

Dimensão	Requisito
Build time	Build incremental deve melhorar, não piorar, com a modularização
Curva de adoção	Engenheiros do monolito devem conseguir contribuir sem treinamento extensivo
Consistência de UI	Componentes visuais compartilhados entre módulos sem criar dependências de UI em módulos de domínio
Backward compatibility	Migração incremental - não é possível parar o desenvolvimento por 6 meses

---

Desafios Ocultos

> **Atenção:** Estes são os pontos onde candidatos com menos senioridade se expõem.

---

① A armadilha da modularização prematura

Propor dezenas de módulos pequenos inspirados em posts de blog de outras empresas é um sinal de alerta. O facilitador da simulação vai pressionar: *quantos módulos você criaria? Como você decide a granularidade?*

Modularização excessiva cria overhead de Gradle, dependências circulares e complexidade de DI que supera os benefícios.

---

② O problema de dependências circulares

Payments precisa de infraestrutura de rede do módulo de messaging. Messaging precisa saber quando uma mensagem contém um pagamento para renderizar o conteúdo certo. Isso cria dependência circular se a separação não for cuidadosa.

A solução geralmente envolve um módulo de contrato (API module pattern).

---

③ Build time pode piorar antes de melhorar

Modularização pode melhorar paralelismo e reaproveitamento de tarefas, mas pode também aumentar configuração, dependências e manutenção. Compare clean e incremental builds com métricas e cargas controladas; melhora de tempo não é garantida nem a longo prazo.

---

④ DI em multi-módulo é substancialmente mais complexo

Hilt/Dagger em multi-módulo tem padrões específicos e não-triviais. Mencionar “vou usar Hilt” sem ter opinião sobre como estruturar a injeção entre módulos é insuficiente para o nível Staff.

---

⑤ **A decisão de não modularizar pode ser a certa**

> Uma alternativa válida é estabelecer fronteiras de código antes de extrair módulos. Escolha o momento da modularização com base em dependências, ownership, builds medidos e custo de manutenção; não há benefício universal.

Perguntas para simulação

Abertura:

> *“Você foi pedido para definir a estratégia de modularização do app. Me diz como você começaria - não a solução, o processo de chegar na solução.”*

Exploração de premissas:

> *“Antes de decidir quantos módulos criar, quais perguntas você precisaria responder sobre o estado atual do codebase e do time?”*

> *“Como você mediria se a modularização foi bem-sucedida? Qual métrica você acompanharia?”*

Pressão técnica:

> *“Payments precisa de autenticação, que hoje está no módulo de messaging. Commerce também vai precisar. Como você estrutura isso sem criar dependência de Commerce em Messaging?”*

> *“Me explica o API module pattern. Quando você usaria e quando não usaria?”*

> *“Como você garantiria que a UI de Payments usa os mesmos componentes visuais do resto do app sem criar dependência de UI no módulo de domínio de Payments?”*

Pressão em processo:

> *“Como você convenceria os 40 engenheiros que trabalham no app hoje a adotar essa nova estrutura? Como você garante que o módulo antigo não continua crescendo enquanto você migra?”*

> *“Qual é o maior risco que você vê nessa estratégia? O que pode dar errado?”*

Liderança:

> *“Você tem 18 meses e um time. Como você divide o trabalho entre migração incremental e desenvolvimento de features novas? Qual é o trade-off?”*

Alternativas e trade-offs

Trade-off central - Modularização agressiva vs. boundary-first approach

Abordagem	Vantagem	Custo
Criar todos os módulos agora com estrutura alvo definida	Direção clara, sem dívida futura de migração	Build complexity imediata, overhead de DI, curva de adoção, pode ser prematuro
Boundaries de código primeiro (packages + interfaces), módulos depois	Incrementalismo seguro, aprendizado antes do comprometimento	Boundaries podem ser violados sem enforcement técnico
Modularização seletiva - apenas domínios com times diferentes	Foco no problema real, não no ideal teórico	Inconsistência arquitetural, messaging continua monolítico

Abordagem possível e limites: a abordagem certa depende de dados que o candidato ainda não tem. A direção seria começar com boundaries de código enforçados por lint rules customizadas, extrair um módulo piloto de baixa dependência (preferencialmente o Design System), aprender com esse processo e só então escalar. Payments seria o segundo módulo prioritário, dado o contexto de compliance e isolamento de domínio financeiro.

> **Dica estratégica:** Em qualquer um dos três desafios, demonstrar que você qualifica o problema antes de propor arquitetura é o sinal mais claro de senioridade Staff. Faça perguntas. Nomeie os trade-offs antes de o facilitador da simulação precisar pedi-los. Seja honesto sobre o que você não sabe.

Contrato de confiabilidade do cenário financeiro

O exercício usa um servidor simulado autoritativo. Uma intenção tem identificador e idempotencyKey estáveis, associados à identidade autenticada e ao payload; reutilizar a chave com outro payload deve ser rejeitado. Deduplicação e registro do efeito precisam ser atômicos no servidor, com retenção de chaves compatível com a janela de retries. O cliente, sozinho, não garante processamento único.

Persistir intenção antes da rede reduz perda de contexto, mas não confirma liquidação. Timeout pode ocorrer após execução: manter estado incerto, consultar por identidade e reconciliar transições monotônicas. Nova tentativa da mesma intenção reutiliza a chave; nova intenção exige confirmação. Validar expiração, cancelamento e autorização antes de retomar trabalho antigo. Histórico offline não autoriza confirmar ou executar pagamento offline.

Notificações são pistas de atualização, não a fonte da verdade. Deduplicar eventos por identificador/versão e buscar estado ao abrir o app e quando houver recuperação permitida. Logs e eventos de demonstração devem omitir chaves Pix reais, valores pessoais e tokens; usar dados sintéticos. Persistência local e retenção exigem minimização, limpeza por conta e proteção adequada; não implicam conformidade legal automática.

Referências técnicas e escopo

Referências de APIs públicas para estudo; não são evidência sobre arquitetura ou processos de empresas reais. Os contratos Kotlin são exemplos educacionais, sem implementação de produtos privados.

- Android: coroutines e ciclo de vida
- Android: WorkRequests e constraints
- Firebase: prioridade de mensagens Android

Career Agent PRO

Edição pública adaptada de saída real. Profundidade técnica, hipóteses explícitas e decisões verificáveis.

www.career-agent-pro.com

support@main.career-agent-pro.com

© 2026 Career Agent PRO. Todos os direitos reservados.

ADAPTED PUBLIC EDITION